LVDT sensor

1. LVDT sensor

Box:- F2

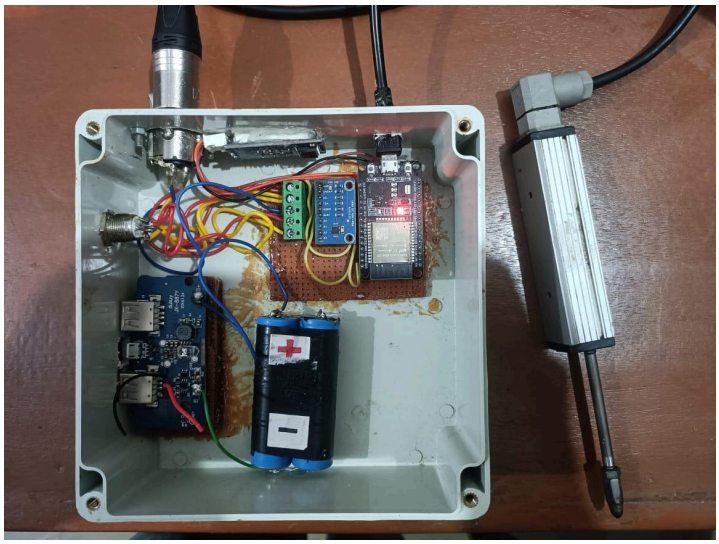
Peripherals:

Board:- ESP32 Dev Module
Wifi ssid = Rector_Bunglow
Wifi password = “Stress1390”
Apikey = “K63IY6550GC3TE8P”

Thinkspeak:

Id = asawari.rawatale03@gmail.com
password = Apple@123

Connections:



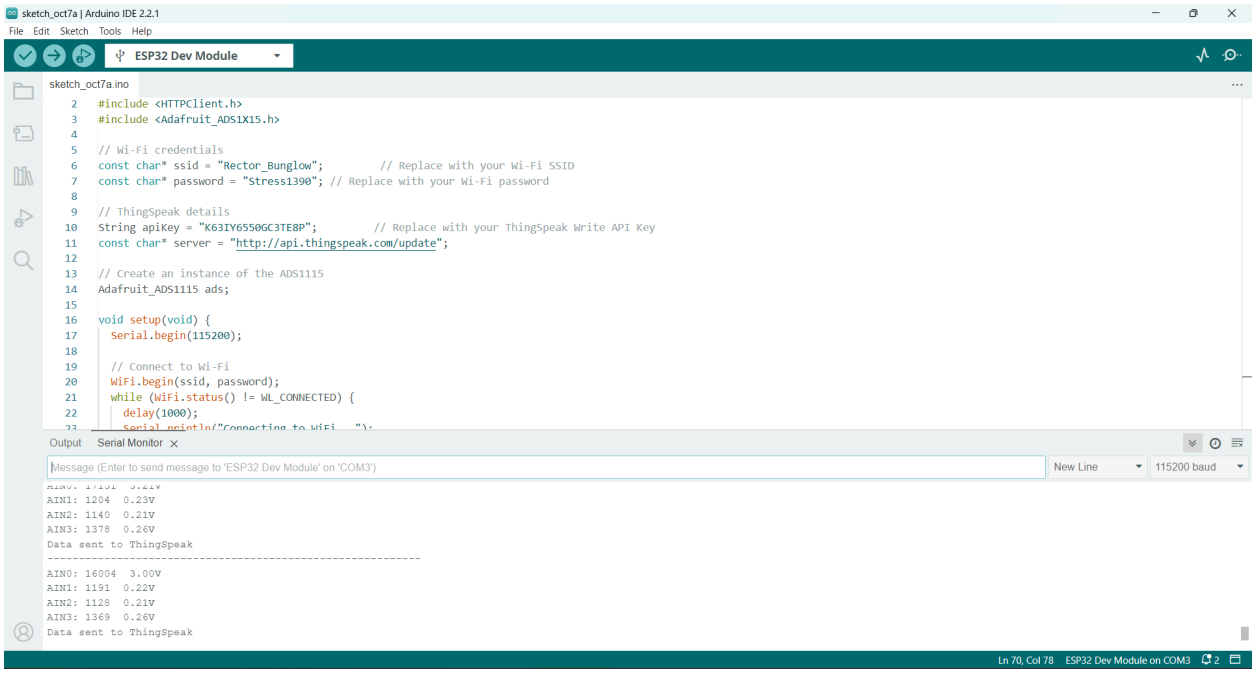
Code:

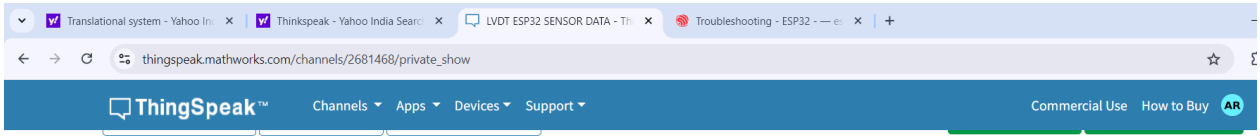
```
#include <WiFi.h>
#include <HTTPClient.h>
#include <Adafruit_ADS1X15.h>

// Wi-Fi credentials
const char* ssid = "Rector_Bunglow";          // Replace with your Wi-Fi
SSID
const char* password = "Stress1390"; // Replace with your Wi-Fi password

// ThingSpeak details
String apiKey = "K63IY6550GC3TE8P";          // Replace with your
ThingSpeak Write API Key
const char* server = "http://api.thingspeak.com/update";

// Create an instance of the ADS1115
Adafruit_ADS1115 ads;

void setup(void) {
  Serial.begin(115200);

  // Connect to Wi-Fi
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected to WiFi");

  // Initialize ADS1115
  if (!ads.begin()) {
    Serial.println("Failed to initialize ADS1115.");
    while (1);
  }

  // You can set the gain if needed. Uncomment the line below for your
desired range:
  // ads.setGain(GAIN_TWOTHIRDS); // +/- 6.144V range
}
```

```
void loop(void) {
  // Read values from ADS1115
  int16_t adc0, adc1, adc2, adc3;
  float volts0, volts1, volts2, volts3;

  adc0 = ads.readADC_SingleEnded(0);
  adc1 = ads.readADC_SingleEnded(1);
  adc2 = ads.readADC_SingleEnded(2);
  adc3 = ads.readADC_SingleEnded(3);

  volts0 = ads.computeVolts(adc0);
  volts1 = ads.computeVolts(adc1);
  volts2 = ads.computeVolts(adc2);
  volts3 = ads.computeVolts(adc3);

  // Print values for debugging

Serial.println("-----------------------------------------------------------
-");
  Serial.print("AIN0: "); Serial.print(adc0); Serial.print("  ");
Serial.print(volts0); Serial.println("V");
  Serial.print("AIN1: "); Serial.print(adc1); Serial.print("  ");
Serial.print(volts1); Serial.println("V");
  Serial.print("AIN2: "); Serial.print(adc2); Serial.print("  ");
Serial.print(volts2); Serial.println("V");
  Serial.print("AIN3: "); Serial.print(adc3); Serial.print("  ");
Serial.print(volts3); Serial.println("V");

  // Send data to ThingSpeak
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;

    // Construct the ThingSpeak update URL with field data
    String url = String(server) + "?api_key=" + apiKey + "&field1=" +
String(volts0) + "&field2=" + String(volts1) + "&field3=" + String(volts2)
+ "&field4=" + String(volts3);

    http.begin(url);
    int httpCode = http.GET();  // Send the GET request to ThingSpeak
```

```
    if (httpCode > 0) {
      String payload = http.getString();  // Get the response from
ThingSpeak
      Serial.println("Data sent to ThingSpeak");
    } else {
      Serial.println("Error sending data to ThingSpeak");
    }

    http.end();  // End the HTTP request
  } else {
    Serial.println("WiFi not connected");
  }

  delay(5000);  // Wait 20 seconds between updates (ThingSpeak free limit
is 15s)
}
```

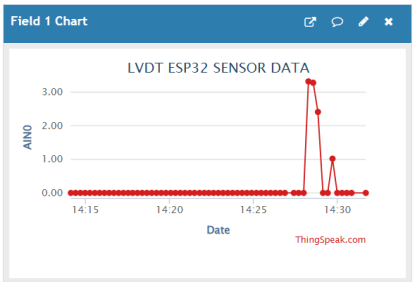
Outputs:

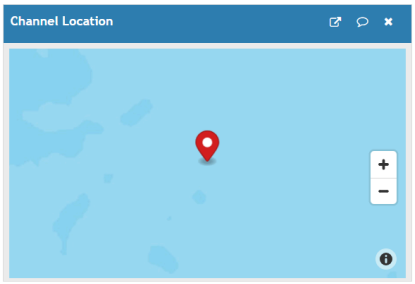




Channel Stats

Created: 3 days ago
Last entry: less than a minute ago
Entries: 314





Calibration:

pending……..

Accelerometer

ACCELEROMETER

Name of the sensor: ACCELEROMETER

Working Principle of the MPU6050: The MPU6050 is a sensor that measures both acceleration (using an accelerometer) and angular velocity (using a gyroscope) along three axes: X, Y, and Z. It uses MEMS technology, where tiny mechanical parts inside the chip respond to motion or rotation. The accelerometer detects changes in position or speed, while the gyroscope detects rotational movement. Both measurements are converted into electrical signals. The sensor has an onboard Digital Motion Processor (DMP) that processes data to calculate motion and orientation. It communicates with other devices using the I2C or SPI protocol and is commonly used in applications like drones, smartphones, and gaming controllers for motion tracking and stabilization.

Methodology:

An accelerometer is constructed using MEMS technology, where a tiny suspended mass (proof mass) is attached to a silicon substrate with micro-springs. When acceleration occurs, the mass moves, causing changes in properties like capacitance (in capacitive types) or voltage (in piezoelectric types). These changes are detected and converted into electrical signals. The chip also includes signal processing circuits for amplification, filtering, and analog-to-digital conversion. The entire structure is housed in a protective casing and communicates with external devices via interfaces like I2C or SPI.

This compact design ensures precision, durability, and versatility for motion sensing.

Specification:

Specification	Details
Sensor Type	3-axis accelerometer and 3-axis gyroscope
Supply Voltage	2.375V to 3.46V
Communication Interface	I2C (up to 400 kHz), SPI (up to 1 MHz)
Accelerometer Range	±2g, ±4g, ±8g, ±16g
Gyroscope Range	±250°/s, ±500°/s, ±1000°/s, ±2000°/s
Sensitivity (Accelerometer)	16384 LSB/g (±2g mode)
Sensitivity (Gyroscope)	131 LSB/°/s (±250°/s mode)
Output Data Rate	Up to 1 kHz
Internal Clock	8 MHz
Temperature Sensor Range	-40°C to +85°C
Temperature Sensor Accuracy	±1°C
Operating Temperature Range	-40°C to +85°C
Power Consumption	<10 µA (sleep mode), 3.6 mA (active mode)
Onboard Processor	Digital Motion Processor (DMP)
Package Dimensions	4 mm x 4 mm x 0.9 mm (QFN package)

Cost:

The cost of the MPU6050 in India ranges from ₹200 to ₹500, depending on the retailer and configuration.

Do's and Don'ts of MPU6050 Accelerometer Sensor:

Do's:

1. Power Supply:
    - Use a stable power source (2.375V to 3.46V) to prevent damage to the sensor.
2. Proper Wiring:

    - Ensure correct connections for I2C (SCL and SDA lines) and verify pull-up resistors are used if necessary.
3. Mounting:
    - Mount the sensor securely to minimize vibrations and noise affecting readings.
4. Calibration:
    - Calibrate the sensor for accurate measurements before use.
5. Signal Filtering:
    - Implement software filters to smooth raw data and reduce noise.
6. Temperature Management:
    - Operate the sensor within the specified temperature range (-40°C to 85°C).

Don'ts:

1. Over-voltage:
    - Do not exceed the maximum supply voltage of 3.46V, as it can permanently damage the sensor.
2. Excessive Stress:
    - Avoid applying mechanical stress or shock beyond its specified limits (e.g., vibrations or impacts).
3. Incorrect Orientation:
    - Do not mount the sensor at odd angles if orientation-based readings are required.
4. Neglect Pull-up Resistors:
    - Avoid omitting pull-up resistors for the I2C lines, as this may cause communication errors.

5. Exposure to Moisture:
    - Do not expose the sensor to moisture or water, as it is not waterproof.

Applications of MPU6050:

1. Motion Tracking:
    - Used in smartphones, tablets, and wearables for orientation and motion sensing.
2. Robotics:
    - Helps in balancing robots, motion control, and navigation.
3. Drones and Quadcopters:
    - Provides orientation, stabilization, and flight control.
4. Gaming Controllers:
    - Detects motion and gestures in gaming devices.
5. Industrial Equipment:
    - Used for vibration monitoring and machine diagnostics.
6. Fitness Trackers:
    - Measures steps, activity levels, and body movements.

Precautions for Using MPU6050:

1. ESD Protection:
    a. Handle the sensor in an electrostatic discharge (ESD)-protected environment.
2. Stable Mounting:

    a. Use anti-vibration mounts for applications involving high-frequency vibrations.
3. Filtering and Smoothing:
    a. Apply digital filters to remove high-frequency noise from data.
4. Proper Calibration:
    a. Regularly calibrate the sensor, especially after mounting or environmental changes.
5. Circuit Protection:
    a. Use a level shifter or logic converter when interfacing with microcontrollers operating at higher voltages.
6. Temperature Compensation:
    a. Account for temperature variations in sensitive applications for better accuracy.
7. Shielding:
    a. Shield the sensor from electromagnetic interference (EMI) if placed near high-frequency devices.

Code for Accelerometer Calibration:-

```
#include "Wire.h"
#include <ESP8266WiFi.h>  // This library is retained for compatibility; Wi-Fi functionality is not used in this version

// MPU6050 I2C address
const uint8_t MPU6050SlaveAddress = 0x68;

// Select GPIO pins for I2C communication (for ESP8266, use GPIO numbers)
const uint8_t scl = 12;  // GPIO12 (D6)
const uint8_t sda = 13;  // GPIO13 (D7)

// Sensitivity scale factors based on the datasheet
const uint16_t AccelScaleFactor = 16384;
```

```
const uint16_t GyroScaleFactor = 131;

// MPU6050 register addresses
const uint8_t MPU6050_REGISTER_SMPLRT_DIV   = 0x19;
const uint8_t MPU6050_REGISTER_PWR_MGMT_1   = 0x6B;
const uint8_t MPU6050_REGISTER_ACCEL_XOUT_H = 0x3B;

int16_t AccelX, AccelY, AccelZ, Temperature, GyroX, GyroY, GyroZ;

unsigned long previousMillis = 0;
const long interval = 100;  // Data update interval in milliseconds

void setup() {
  Serial.begin(115200);
  Wire.begin(sda, scl);  // Initialize I2C with the correct GPIO pins
  MPU6050_Init();

  // Display initial message
  Serial.println("MPU6050 data visualization on the Serial Plotter:");
}

void loop() {
  unsigned long currentMillis = millis();

  // Only update data every interval milliseconds
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;  // Save the last time data was
updated

    // Fetch and display MPU6050 data
    fetchAndDisplayData();
  }
}

void fetchAndDisplayData() {
  double Ax, Ay, Az, T, Gx, Gy, Gz;

  // Read sensor values
  Read_RawValue(MPU6050SlaveAddress, MPU6050_REGISTER_ACCEL_XOUT_H);
```

```
  // Convert raw values to human-readable units
  Ax = (double)AccelX / AccelScaleFactor;
  Ay = (double)AccelY / AccelScaleFactor;
  Az = (double)AccelZ / AccelScaleFactor;
  T = (double)Temperature / 340.00 + 36.53; // Temperature formula
  Gx = (double)GyroX / GyroScaleFactor;
  Gy = (double)GyroY / GyroScaleFactor;
  Gz = (double)GyroZ / GyroScaleFactor;

  // Serial Plotter output
  Serial.print(Ax); Serial.print(" ");
  Serial.print(Ay); Serial.print(" ");
  Serial.print(Az); Serial.print(" ");
  Serial.print(T); Serial.print(" ");
  Serial.print(Gx); Serial.print(" ");
  Serial.print(Gy); Serial.print(" ");
  Serial.println(Gz);
}

// Function to write to MPU6050 register
void I2C_Write(uint8_t deviceAddress, uint8_t regAddress, uint8_t data) {
  Wire.beginTransmission(deviceAddress);
  Wire.write(regAddress);
  Wire.write(data);
  Wire.endTransmission();
}

// Function to read all 14 bytes from the sensor
void Read_RawValue(uint8_t deviceAddress, uint8_t regAddress) {
  Wire.beginTransmission(deviceAddress);
  Wire.write(regAddress);
  Wire.endTransmission();
  Wire.requestFrom(deviceAddress, (uint8_t)14);

  AccelX = (((int16_t)Wire.read() << 8) | Wire.read());
  AccelY = (((int16_t)Wire.read() << 8) | Wire.read());
  AccelZ = (((int16_t)Wire.read() << 8) | Wire.read());
  Temperature = (((int16_t)Wire.read() << 8) | Wire.read());
  GyroX = (((int16_t)Wire.read() << 8) | Wire.read());
  GyroY = (((int16_t)Wire.read() << 8) | Wire.read());
```

```
  GyroZ = (((int16_t)Wire.read() << 8) | Wire.read());
}

// Function to initialize the MPU6050
void MPU6050_Init() {
  delay(150);  // Delay for stabilization
  I2C_Write(MPU6050SlaveAddress, MPU6050_REGISTER_SMPLRT_DIV, 0x07);
  I2C_Write(MPU6050SlaveAddress, MPU6050_REGISTER_PWR_MGMT_1, 0x01);
}
```

Code to plot the reading on thinkspeak:

```
#include "Wire.h"
#include <ESP8266WiFi.h>
#include <WiFiClient.h>

// Wi-Fi credentials
const char* ssid = "Rector_Bunglow";  // Replace with your Wi-Fi SSID
const char* password = "Stress1390";  // Replace with your Wi-Fi password

// ThingSpeak API Key (Write API Key)
const char* apiKey = "F5VY3XER253G086B";  // Replace with your ThingSpeak
Write API Key
const char* server = "api.thingspeak.com"; // ThingSpeak server

// MPU6050 I2C address
const uint8_t MPU6050SlaveAddress = 0x68;

// Select GPIO pins for I2C communication (for ESP8266, use GPIO numbers)
const uint8_t scl = 12;  // GPIO12 (D6)
const uint8_t sda = 13;  // GPIO13 (D7)

// Sensitivity scale factors based on the datasheet
const uint16_t AccelScaleFactor = 16384;
const uint16_t GyroScaleFactor = 131;

// MPU6050 register addresses
const uint8_t MPU6050_REGISTER_SMPLRT_DIV   = 0x19;
```

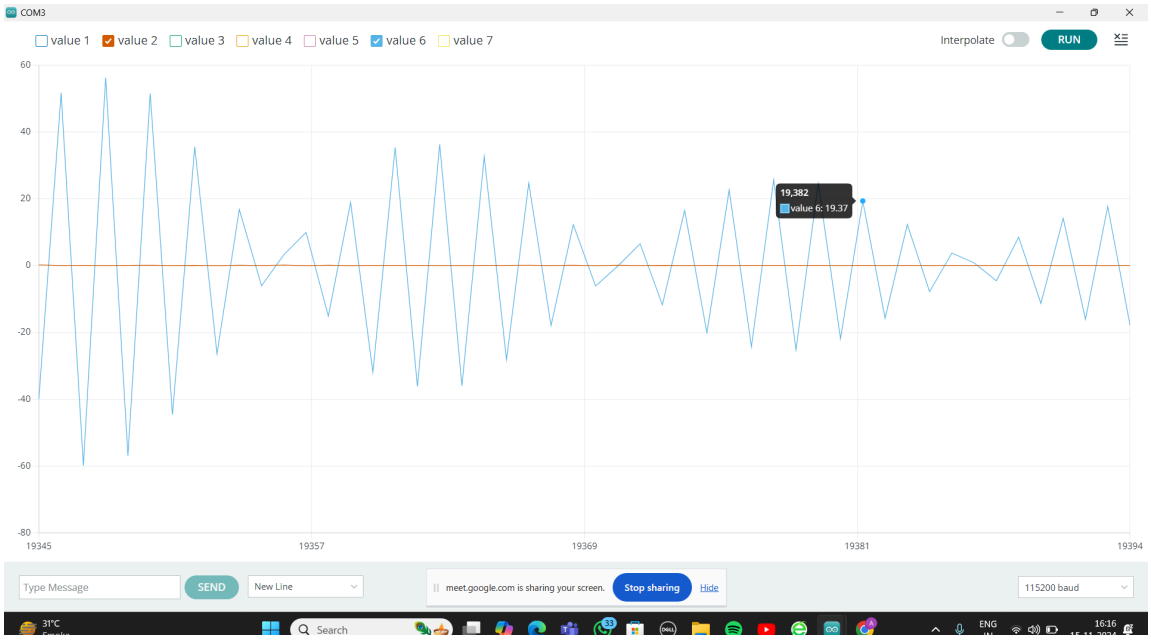
```
const uint8_t MPU6050_REGISTER_PWR_MGMT_1   = 0x6B;
const uint8_t MPU6050_REGISTER_ACCEL_XOUT_H = 0x3B;

int16_t AccelX, AccelY, AccelZ, Temperature, GyroX, GyroY, GyroZ;

unsigned long previousMillis = 0;
const long interval = 250;  // Data update interval in milliseconds

void setup() {
  Serial.begin(115200);
  Wire.begin(sda, scl);  // Initialize I2C with the correct GPIO pins
  MPU6050_Init();

  // Connecting to Wi-Fi
  Serial.println();
  Serial.print("Connecting to WiFi");
  WiFi.begin(ssid, password);

  // Wait until connected to Wi-Fi
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("Connected to WiFi!");
  Serial.println("IP Address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  unsigned long currentMillis = millis();

  // Only send data every interval milliseconds
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;  // Save the last time data was sent

    // Fetch and display MPU6050 data
    fetchAndDisplayData();
  }
```

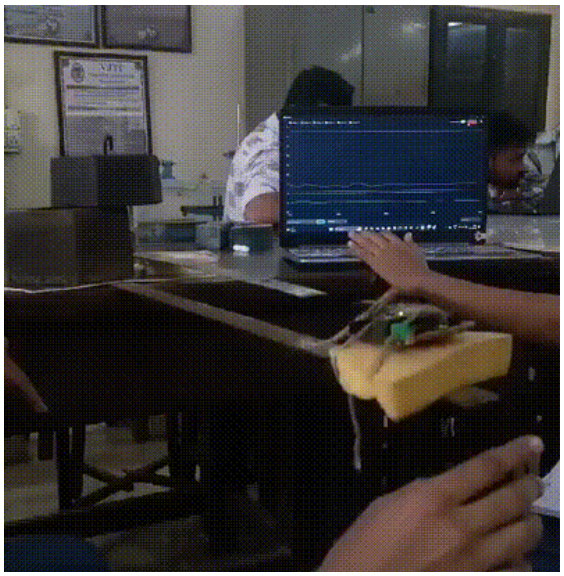
```
}

void fetchAndDisplayData() {
  double Ax, Ay, Az, T, Gx, Gy, Gz;

  // Read sensor values
  Read_RawValue(MPU6050SlaveAddress, MPU6050_REGISTER_ACCEL_XOUT_H);

  // Convert raw values to human-readable units
  Ax = (double)AccelX / AccelScaleFactor;
  Ay = (double)AccelY / AccelScaleFactor;
  Az = (double)AccelZ / AccelScaleFactor;
  T = (double)Temperature / 340.00 + 36.53; // Temperature formula
  Gx = (double)GyroX / GyroScaleFactor;
  Gy = (double)GyroY / GyroScaleFactor;
  Gz = (double)GyroZ / GyroScaleFactor;

  // Serial Monitor output
  Serial.println("==== MPU6050 Sensor Data ====");
  Serial.print("Ax: "); Serial.print(Ax);
  Serial.print(" Ay: "); Serial.print(Ay);
  Serial.print(" Az: "); Serial.print(Az);
  Serial.print(" T: "); Serial.print(T);
  Serial.print(" Gx: "); Serial.print(Gx);
  Serial.print(" Gy: "); Serial.print(Gy);
  Serial.print(" Gz: "); Serial.println(Gz);

  // Serial Plotter output
  Serial.print(Ax); Serial.print(" ");
  Serial.print(Ay); Serial.print(" ");
  Serial.print(Az); Serial.print(" ");
  Serial.print(T); Serial.print(" ");
  Serial.print(Gx); Serial.print(" ");
  Serial.print(Gy); Serial.print(" ");
  Serial.println(Gz);

  // Send data to ThingSpeak
  sendToThingSpeak(Ax, Ay, Az, T, Gx, Gy, Gz);
}
```

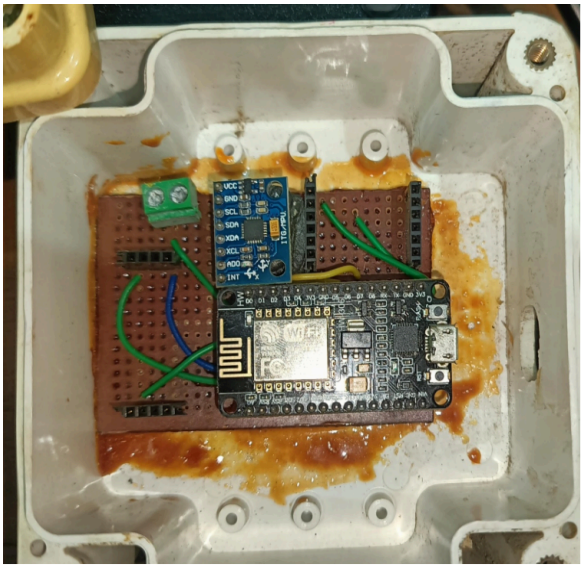
```
// Function to write to MPU6050 register
void I2C_Write(uint8_t deviceAddress, uint8_t regAddress, uint8_t data) {
  Wire.beginTransmission(deviceAddress);
  Wire.write(regAddress);
  Wire.write(data);
  Wire.endTransmission();
}

// Function to read all 14 bytes from the sensor
void Read_RawValue(uint8_t deviceAddress, uint8_t regAddress) {
  Wire.beginTransmission(deviceAddress);
  Wire.write(regAddress);
  Wire.endTransmission();
  Wire.requestFrom(deviceAddress, (uint8_t)14);

  AccelX = (((int16_t)Wire.read() << 8) | Wire.read());
  AccelY = (((int16_t)Wire.read() << 8) | Wire.read());
  AccelZ = (((int16_t)Wire.read() << 8) | Wire.read());
  Temperature = (((int16_t)Wire.read() << 8) | Wire.read());
  GyroX = (((int16_t)Wire.read() << 8) | Wire.read());
  GyroY = (((int16_t)Wire.read() << 8) | Wire.read());
  GyroZ = (((int16_t)Wire.read() << 8) | Wire.read());
}

// Function to initialize the MPU6050
void MPU6050_Init() {
  delay(150);  // Delay for stabilization
  I2C_Write(MPU6050SlaveAddress, MPU6050_REGISTER_SMPLRT_DIV, 0x07);
  I2C_Write(MPU6050SlaveAddress, MPU6050_REGISTER_PWR_MGMT_1, 0x01);
}

// Function to send data to ThingSpeak
void sendToThingSpeak(double Ax, double Ay, double Az, double T, double
Gx, double Gy, double Gz) {
  WiFiClient client;

  if (client.connect(server, 80)) {
    String postData = String("api_key=") + apiKey +
                      "&field1=" + String(Ax) +
                      "&field2=" + String(Ay) +
```

```
                        "&field3=" + String(Az) +
                        "&field4=" + String(T) +
                        "&field5=" + String(Gx) +
                        "&field6=" + String(Gy) +
                        "&field7=" + String(Gz);

    client.println("POST /update HTTP/1.1");
    client.println("Host: api.thingspeak.com");
    client.println("Connection: close");
    client.println("Content-Type: application/x-www-form-urlencoded");
    client.print("Content-Length: ");
    client.println(postData.length());
    client.println();
    client.println(postData);

    // Check for a response from ThingSpeak
    while (client.available()) {
      String response = client.readString();
      Serial.println(response);
    }

    Serial.println("Data sent to ThingSpeak.");
  } else {
    Serial.println("Failed to connect to ThingSpeak.");
  }
  client.stop();
}
```

Outputs:







Calibration Details :-

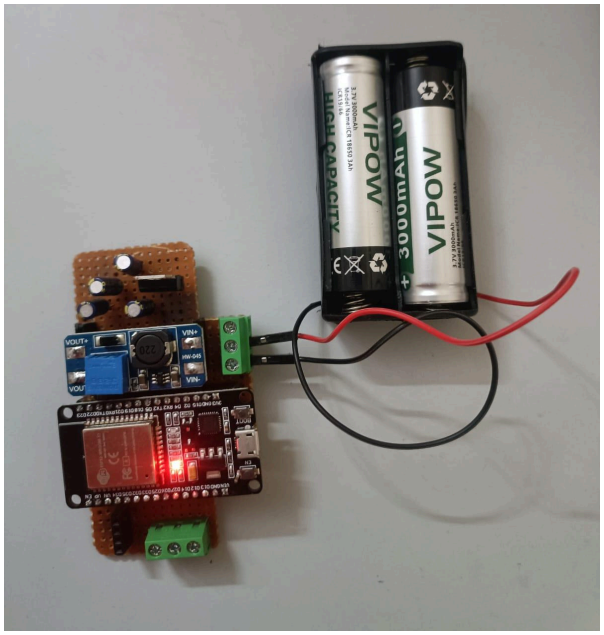
Counter weights :- 17Kg
Scale weights :-
Time delay between peaks :- 200ms
Oscillations/sec = 5

Temperature

Temperature Sensor (BME280)

Sensor new design:-

This circuit is designed to provide a battery backup for this sensor and also to fit in two batteries In the box:



Code:-

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
#include <WiFi.h>
#include <HTTPClient.h>

// Replace with your network credentials
const char* ssid = "Rector_Bunglow";
const char* password = "Stress1390";

// ThingSpeak API information
const char* server = "api.thingspeak.com";
const char* apiKey = "F5VY3XER253G086B";

// BME280 I2C address (default is 0x76 or 0x77)
#define BME280_ADDRESS 0x76
Adafruit_BME280 bme;

// Timing variables
unsigned long previousMillis = 0;
const long interval = 10000;  // Data update interval in milliseconds
(e.g., 10 seconds)

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  // Connect to WiFi
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");

  // Initialize BME280 sensor
  if (!bme.begin(BME280_ADDRESS)) {
    Serial.println("Could not find a valid BME280 sensor, check wiring!");
```

```
    while (1);
  }

  Serial.println("BME280 data visualization on the Serial Plotter:");
}

void loop() {
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;  // Save the last time data was
updated

    // Read temperature from the BME280 sensor
    float temperature = bme.readTemperature();

    // Print temperature to Serial Plotter
    Serial.println(temperature);

    // Send temperature data to ThingSpeak
    if (WiFi.status() == WL_CONNECTED) {
      HTTPClient http;
      String url = String(server) + "?api_key=" + apiKey + "&field1=" +
String(temperature);

      http.begin(url);  // Start HTTP connection
      int httpCode = http.GET();  // Send GET request

      if (httpCode > 0) {
        Serial.println("Data sent to ThingSpeak successfully");
      } else {
        Serial.println("Error in sending data to ThingSpeak");
      }

      http.end();  // Close HTTP connection
    } else {
      Serial.println("WiFi not connected");
    }
  }
}
```

Outputs:

Calibration:

pending…..